

Flutter Take-Home Assignment

Estimated Time: 4 hours (you can take 1-2 days to come back)

Objective

Build a small shopping application using Flutter. The goal of this assignment is to demonstrate your ability to:

- Consume REST APIs
 - Manage application state
 - Structure and organize code
 - Handle edge cases and failures
 - Build a polished user experience
-

API

Use the following API: <https://fakestoreapi.com/products>

Endpoints

- **Get all products** GET /products
 - **Get product details** GET /products/{id}
-

Required Features

1. Product Listing Screen

Display a list of products fetched from the API.

Each product item should display:

- Product image
- Product title
- Product price
- Product rating

Functionality

- Fetch products from the API when the app launches

- Pull-to-refresh support
 - Search products by title
 - Loading state while data is being fetched
 - Empty state when no products match the search criteria
 - Error state when API calls fail
 - Retry functionality from the error state
-

2. Product Details Screen

Selecting a product should navigate to a dedicated details screen.

Display:

- Product image
- Product title
- Product description
- Product category
- Product price
- Product rating

Functionality

- Add product to favorites
 - Remove product from favorites
 - Favorite status should be reflected consistently throughout the application
-

3. Favorites Screen

Create a separate screen that displays all favorited products.

Functionality

- View all favorite products
 - Remove products from favorites
 - Favorite products should persist after restarting the application
-

Technical Requirements

- Use Flutter and Dart
- Consume data from the provided API

- Manage application state appropriately
 - Handle loading, success, empty, and error states
 - Persist favorite products locally
 - Ensure the application runs on both Android and iOS
 - Write clean, readable, and maintainable code
 - Use packages only when they provide clear value
-

Expected User Experience

The application should feel complete and polished.

Users should be able to:

- Browse products immediately after launch
 - Search products easily
 - View detailed product information
 - Add and remove favorites seamlessly
 - Reopen the application and retain favorite selections
 - Recover gracefully from network failures
-

Bonus (Optional)

Any of the following:

- Product sorting (price, rating, etc.)
 - Additional UX improvements
-

Submission

Please share:

- Source code repository
- Instructions to run the application
- Any assumptions or trade-offs made during development